

Xiaomi AI Speaker L07A: повна інструкція з UART-доступу, root shell та заміни китайських голосових звуків

0. Що було зроблено

У межах цієї процедури на Xiaomi / Redmi AI Speaker L07A було успішно:

- знайдено штатний UART debug-роз'єм;
- підключенося до консолі через USB-TTL адаптер;
- зупинено U-Boot;
- тимчасово завантажено Linux з `init=/bin/sh`;
- отримано root shell без підбору DSA-пароля;
- підтверджено, що rootfs є read-only squashfs;
- знайдено writable EXT4-розділ `/dev/nand0p9`, який монтується як UDISK;
- знайдено активний symlink звукового пакета `/tmp/udisk/sound`;
- витягнуто оригінальний `sound-vendor`;
- згенеровано кастомний sound pack без китайських голосових фраз;
- залито кастомні звуки через UART як shell-скрипт;
- перемкнуто `/tmp/udisk/sound` на `/tmp/udisk/sound_custom`;
- успішно перевірено, що китайські голосові фрази прибрано/замінено без перепрошивки rootfs.

Головна ідея методу: не чіпати підписаний rootfs і не перепакувувати NAND-образи, а використати штатний writable-розділ UDISK і symlink, який уже використовується системою для вибору активного sound pack.

1. Дисклеймер

Усе нижче робиться на власний ризик.

Ця процедура передбачає фізичний доступ до плати, підключення UART, зупинку U-Boot і ручне виконання команд у root shell. Помилки в командах типу `erase`, `saveenv`, `dd`, `nandwrite`, `flash_erase`, `rm` не там, де треба, можуть пошкодити прошивку або перетворити пристрій на "цеглу".

Безпечна частина цього методу полягає в тому, що ми:

- не пишемо в rootfs;
- не перепакуємо squashfs;
- не прошиваємо NAND;
- не змінюємо U-Boot env через `saveenv`;
- не чіпаємо secure boot;
- працюємо тільки з writable-розділом `/dev/nand0p9`.

Не публікуйте:

- повні boot logs із серійниками;
 - `TOKEN` з UDISK;
 - `magic[release]`;
 - NAND/rootfs dumps;
 - оригінальні Xiaomi audio assets;
 - приватні ключі, токени, device-specific дані.
-

2. Що потрібно

Hardware

Потрібно:

- Xiaomi / Redmi AI Speaker L07A;
- лабораторний блок живлення або штатне 12V живлення;
- USB-TTL UART адаптер з логічним рівнем 3.3V;
- JST-XH 2.54 3-pin кабель або сумісний роз'єм;
- ПК з Windows;
- Tera Term 5 або інший UART terminal;
- Git, якщо планується оформлення результату в репозиторій.

Важливо: USB-TTL адаптер має працювати саме на 3.3V logic. Не використовувати RS232. Не подавати 5V/3V3 з адаптера на плату.

Software

Було використано:

- Tera Term 5;
 - PowerShell;
 - Git;
 - локальні helper-файли:
 - `sound-vendor-system-melodies.zip`;
 - `l07a_upload_system_sounds_AiNiRobot.sh`;
 - `README.md`;
 - `LICENSE`.
-

3. UART pinout

На платі L07A знайдено штатний 3-pin JST-XH 2.54 debug connector.

Підпис на платі:

1. `W / TX`
2. `B / GND`

3. G / RX

Підключення до USB-TTL:

Board TX -> USB-TTL RX
Board RX -> USB-TTL TX
Board GND -> USB-TTL GND

Не підключати:

USB-TTL VCC
USB-TTL 3V3
USB-TTL 5V

Живлення колонки подавати окремо через штатний 12V input або лабораторний БЖ.

Рекомендовано для першого запуску на лабораторнику:

12.0 V
current limit: 0.5–1.0 A

Якщо пристрій при старті впирається в обмеження струму, можна підняти ліміт до приблизно 1.2 A.

4. Налаштування Tera Term

У Tera Term:

Serial port
Speed: 115200
Data: 8 bit
Parity: none
Stop bits: 1 bit
Flow control: none

Тобто:

115200 8N1, no flow control

Для логування:

File -> Log...

Для передачі shell-скрипта пізніше:

File -> Send file...

Параметри Send file для текстового shell-скрипта:

Bulk read: enabled
Binary: disabled
Delay type: per line або line delay
Delay time: 5–10 ms

5. Перший UART boot log

Після підключення UART і подачі 12V на пристрій у Tera Term має з'явитися boot log.

Важливі ознаки:

CPU: Allwinner Family
Model: sun8iw18
DRAM: 64 MiB
secure enable bit: 1
U-Boot 2018.05
Build: jenkins-Mico_l07a_ota_publish-63

При звичайному завантаженні Linux доходить до login prompt:

L07A login:

Спроби `root`, `admin`, `xiaomi` не дають нормального shell, бо login захищений кастоною Xiaomi/Mico DSA-перевіркою.

У логах видно характерні рядки:

```
try get user:magic[release]  
dsa verify sign  
DSA_verify  
dsa verify failed
```

Це означає, що простий пароль не використовується. Потрібно заходити не через Linux login, а через U-Boot.

6. Зупинка U-Boot

Під час старту з'являється:

Hit any key to stop autoboot: 0

Потрібно швидко натиснути клавішу в Tera Term.

Якщо вдалося, з'явиться U-Boot prompt:

=>

Безпечні команди для розвідки:

help

version

printenv

Не вводити:

saveenv

erase

sunxi_flash erase

sunxi_flash write

fastboot

efex

uburn

nand erase

flash_erase

7. Корисний U-Boot env

Команда:

printenv

Важливі змінні:

boot_fastboot=fastboot

boot_first=sunxi_flash read 40007800 \${boot_partition_1};bootm 40007800

boot_partition_1=kernel1

boot_partition_2=kernel2

boot_second=sunxi_flash read 40007800 \${boot_partition_2};bootm 40007800

bootcmd=run setargs_first boot_first

bootdelay=0

console=ttyS0,115200

earlyprintk=sunxi-uart,0x05000000

first_root=/dev/nand0p3

second_root=/dev/nand0p5

init=/sbin/init

rdinit=/rdinit

partitions=env@nand0p1:kernel1@nand0p2:rootfs1@nand0p3:kernel2@nand0p4:rootfs2@nand0p5:misc@nand0p6:privat

rotpk_status=1

Активна система:

kernel1 + rootfs1

Активний root:

/dev/nand0p3

8. Тимчасовий root shell через init=/bin/sh

Зміни нижче тимчасові. Вони діють тільки для поточного boot, якщо не виконувати `saveenv`.

У U-Boot:

run setargs_first

Перевірити сформовані bootargs:

printenv bootargs

Заміна init на shell:

```
setenv bootargs earlyprintk=${earlyprintk} console=${console} root=${first_root} rootwait init=/bin/sh  
rdinit=${rdinit} loglevel=8 partitions=${partitions} gpt=${gpt} rotpk_status=${rotpk_status}
```

Перевірити:

printenv bootargs

Має бути:

init=/bin/sh

Запуск ядра:

run boot_first

Після завантаження Linux очікувано з'являється:

```
BusyBox v1.27.2 () built-in shell (ash)
```

```
/bin/sh: can't access tty; job control turned off
```

Prompt:

```
/ #
```

Перевірка root:

```
id
```

Очікувано:

```
uid=0(root) gid=0(root)
```

9. Підняття /proc та /sys

Оскільки ми зайшли напямую через `init=/bin/sh`, нормальний `/sbin/init` не стартував. Тому `/proc` і `/sys` потрібно змонтувати вручну.

```
mount -t proc proc /proc
```

```
mount -t sysfs sysfs /sys
```

Спроба змонтувати `/dev` може дати `Resource busy`. Це нормально:

```
mount -t devtmpfs devtmpfs /dev
```

Перевірка cmdline:

```
cat /proc/cmdline
```

Очікувано має бути щось на кшталт:

```
earlyprintk=sunxi-uart,0x05000000 console=ttyS0,115200 root=/dev/nand0p3 rootwait init=/bin/sh  
rdinit=/rdinit loglevel=8 partitions=... gpt=1 rotpk_status=1
```

10. Перевірка NAND partitions

Список NAND block devices:

```
ls -la /dev/nand*
```

Очікувано:

```
/dev/nand0  
/dev/nand0p1  
/dev/nand0p2  
/dev/nand0p3  
/dev/nand0p4  
/dev/nand0p5  
/dev/nand0p6  
/dev/nand0p7  
/dev/nand0p8  
/dev/nand0p9
```

Розміри розділів:

```
cat /proc/partitions
```

Побачена карта:

```
nand0p1 — 256 KB — env  
nand0p2 — 6 MB — kernel1  
nand0p3 — 36 MB — rootfs1  
nand0p4 — 6 MB — kernel2  
nand0p5 — 36 MB — rootfs2  
nand0p6 — 256 KB — misc  
nand0p7 — 256 KB — private  
nand0p8 — 256 KB — crashlog  
nand0p9 — about 26 MB — UDISK
```

Перевірка mounts:

```
mount
```

Було:

```
/dev/root on / type squashfs (ro,relatime)
```

Це означає, що rootfs read-only. Прямо писати в `/usr/share/...` не можна.

11. Пошук звукових файлів

Пошук WAV:

```
find / -iname "*.wav" 2>/dev/null
```

Пошук OPUS:

```
find / -iname "*.opus" 2>/dev/null
```


Знайдені оригінальні sound packs:

`/usr/share/sound-vendor/AiNiRobot`

`/usr/share/sound-vendor/XiaoMi`

`/usr/share/sound-vendor/XiaoMi_M88`

Приклади файлів:

`/usr/share/sound-vendor/AiNiRobot/first_voice.opus`
`/usr/share/sound-vendor/AiNiRobot/bluetooth_connect.opus`
`/usr/share/sound-vendor/AiNiRobot/bluetooth_disconnect.opus`
`/usr/share/sound-vendor/AiNiRobot/reset.opus`
`/usr/share/sound-vendor/AiNiRobot/reset_wait.opus`
`/usr/share/sound-vendor/AiNiRobot/wakeup_wozai.wav`

Список активного паку:

`ls -lah /usr/share/sound-vendor/AiNiRobot`

Особливо цікаві файли:

`first_voice.opus`
`enter_config_mode.opus`
`init_wifi_config.opus`
`init_wifi_success.opus`
`reset.opus`
`reset_wait.opus`
`wakeup_.wav`
`bluetooth_.opus`
`wifi_*.opus`

12. Монтуювання UDISK

Створити mount point:

`mkdir -p /tmp/udisk`

Змонтувати `/dev/nand0p9`:

`mount -o rw /dev/nand0p9 /tmp/udisk`

Очікувані kernel messages:

EXT4-fs (nand0p9): couldn't mount as ext3 due to feature incompatibilities
EXT4-fs (nand0p9): couldn't mount as ext2 due to feature incompatibilities

EXT4-fs (nand0p9): recovery complete
EXT4-fs (nand0p9): mounted filesystem with ordered data mode

Це нормально. Розділ змонтувався як EXT4.

Перевірка:

ls -la /tmp/udisk

Серед вмісту знайдено:

TOKEN
ai-crontab
alarm
bt
chimehourly
console
etc
log
mibrain
miio
notify
player
sound
status
wifi

Найважливіше:

/tmp/udisk/sound -> /usr/share/sound-vendor/AiNiRobot

Перевірити:

ls -la /tmp/udisk/sound

Було:

lrwxrwxrwx /tmp/udisk/sound -> /usr/share/sound-vendor/AiNiRobot

Це ключове відкриття. Активний sound pack задається symlink на writable UDISK.

13. Чому не треба патчити rootfs

Rootfs:

/dev/root on / type squashfs (ro,relatime)

U-Boot перед стартом показує верифікацію:

```
pubkey rootfs1 valid
partition rootfs1 verify pass
```

Тому пряме перепакування rootfs може зламати boot через secure verification.

Безпечний шлях:

- не чіпати `/usr/share/sound-vendor`;
 - не патчити squashfs;
 - не прошивати NAND;
 - створити новий sound pack у `/tmp/udisk/sound_custom`;
 - перемкнути `/tmp/udisk/sound` на кастомну папку.
-

14. Підготовка кастомного sound pack

Було створено системний sound pack без голосу:

`packs/sound-vendor-system-melodies.zip`

А також upload-скрипт:

`scripts/I07a_upload_system_sounds_AiNiRobot.sh`

Скрипт створює:

`/tmp/udisk/sound_custom`

і записує туди файли з такими самими назвами, як у активному паку `AiNiRobot`.

Структура має бути плоска:

```
/tmp/udisk/sound_custom/first_voice.opus
/tmp/udisk/sound_custom/wakeup_wozai.wav
/tmp/udisk/sound_custom/bluetooth_connect.opus
/tmp/udisk/sound_custom/reset.opus
```

Не так:

```
/tmp/udisk/sound_custom/AiNiRobot/first_voice.opus
```

Тобто symlink має вести саме на папку, де файли лежать напряму.

15. Чому не використовувався wget

У BusyBox були знайдені:

```
/usr/bin/nc  
/bin/wget
```

Але в emergency shell через `init=/bin/sh` нормальна система не стартувала, тому Wi-Fi/network services не були підняті. UART не блокує мережу сам по собі; мережі не було через те, що ми обійшли `/sbin/init`.

Тому практичний upload було зроблено через UART.

16. Перевірка printf перед UART upload

Щоб переконатися, що shell може створювати binary-файли через escape-послідовності:

```
printf '\x41\x42\x43' > /tmp/printf_test.bin
```

```
hexdump -C /tmp/printf_test.bin
```

Очікувано:

```
00000000 41 42 43 |ABC|
```

Це підтвердило, що upload-скрипт може створювати файли через `printf`.

17. Заливання upload-скрипта через UART

На L07A:

```
cat > /tmp/udisk/l07a_upload_system_sounds_AiNiRobot.sh
```

Після цього Tera Term переходить у режим очікування вводу для `cat`.

У Tera Term:

File -> Send file...

Вибрати файл:

`l07a_upload_system_sounds_AiNiRobot.sh`

Налаштування:

Bulk read: enabled

Binary: disabled

Delay per line: 5-10 ms

Після завершення передачі натиснути:

Ctrl+D

Це завершує `cat` і повертає prompt `/ #`.

Перевірити розмір:

```
ls -lh /tmp/udisk/l07a_upload_system_sounds_AiNiRobot.sh
```

Успішний приклад:

```
-rw-r--r-- 1 root root 436.0K /tmp/udisk/l07a_upload_system_sounds_AiNiRobot.sh
```

18. Запуск upload-скрипта

Запустити:

```
sh /tmp/udisk/l07a_upload_system_sounds_AiNiRobot.sh
```

Очікуваний лог:

```
[L07A] creating /tmp/udisk/sound_custom
[L07A] writing system sound files...
[L07A] write bluetooth_already_connected.opus
[L07A] write bluetooth_connect.opus
[L07A] write bluetooth_disconnect.opus
[L07A] write first_voice.opus
[L07A] write reset.opus
[L07A] write wakeup_wozai.wav
...
[L07A] DONE. Custom sounds are in: /tmp/udisk/sound_custom
```

Перевірити:

```
ls -la /tmp/udisk/sound_custom | head
```

```
ls -la /tmp/udisk/sound_custom/first_voice.opus
```

```
ls -la /tmp/udisk/sound_custom/wakeup_wozai.wav
```

19. Перемикання активного sound pack

Початковий стан:

```
/tmp/udisk/sound -> /usr/share/sound-vendor/AiNiRobot
```

Зробити backup symlink:

```
mv /tmp/udisk/sound /tmp/udisk/sound.factory
```

Створити новий symlink:

```
ln -s /tmp/udisk/sound_custom /tmp/udisk/sound
```

Записати зміни:

```
sync
```

Перевірити:

```
ls -la /tmp/udisk/sound
```

Очікувано:

```
/tmp/udisk/sound -> /tmp/udisk/sound_custom
```

Перевірити файли через symlink:

```
ls -la /tmp/udisk/sound/first_voice.opus
```

```
ls -la /tmp/udisk/sound/wakeup_wozai.wav
```

Якщо файли видно, symlink працює.

20. Перезавантаження

Спробувати:

```
reboot
```

Якщо reboot із emergency shell не спрацьовує, просто зробити power-cycle через 12V живлення.

Після старту перевірити:

- чи зник китайський стартовий голос;
- чи замість `first_voice.opus` звучить короткий системний сигнал;
- чи wakeup-звуки замінені;
- чи Bluetooth / Wi-Fi / reset prompt-и не падають;
- чи система загалом стартує штатно.

Підтверджений результат: китайські голосові "барини" було прибрано/замінено на системні мелодії без перепрошивки rootfs.

21. Відкат

Якщо кастомні звуки не сподобались або щось пішло не так, знову зайти через UART/U-Boot/root shell, змонтувати UDISK:

```
mkdir -p /tmp/udisk
```

```
mount -o rw /dev/nand0p9 /tmp/udisk
```

Повернути старий symlink:

```
rm /tmp/udisk/sound
```

```
mv /tmp/udisk/sound.factory /tmp/udisk/sound
```

```
sync
```

Перевірити:

```
ls -la /tmp/udisk/sound
```

Очікувано:

```
/tmp/udisk/sound -> /usr/share/sound-vendor/AiNiRobot
```

Після цього reboot або power-cycle.

22. Витягнення оригінального sound-vendor

Оригінальний sound pack можна витягнути для аналізу, але не варто публікувати його в репозиторії.

Було створено архів у RAM:

```
mount -t tmpfs tmpfs /tmp
```

```
tar -czf /tmp/sound-vendor.tar.gz -C /usr/share sound-vendor
```

Перевірка:

```
ls -lh /tmp/sound-vendor.tar.gz
```

Розмір у цьому випадку:

2141492 bytes

Через UART великий дамп одним шматком виявився ненадійним: Tera Term/UART міг губити або вставляти символи. Робочий варіант — дамп чанками через `dd` + `hexdump`.

Приклад chunk:

```
dd if=/tmp/sound-vendor.tar.gz bs=1024 skip=0 count=512 2>/dev/null | hexdump -v -e '16/1 "%02x"
"\n"'
```

Але для фінального методу витягнення всього vendor pack не є обов'язковим. Достатньо мати кастомний pack і upload-скрипт.

23. Оформлення Git repository

Рекомендована структура:

```
README.md
LICENSE
.gitignore
scripts/I07a_upload_system_sounds_AiNiRobot.sh
packs/sound-vendor-system-melodies.zip
docs/UKRAINIAN_FULL_GUIDE.md
```

Не додавати в репозиторій:

```
.log
.dump
.bin
.img
*.tar.gz
оригінальні Xiaomi sounds
NAND dumps
rootfs dumps
TOKEN
серійники
magic/release-specific дані
```

Приклад `.gitignore`:

```
.log
.tmp
.bin
.img
.dump
.tar.gz
puscache/
.DS_Store
Thumbs.db
sound-vendor-full-extracted/
sound-vendor/
```


.decoded
.raw
**.hex*

24. Git commands

Ініціалізація:

```
git init
```

Додавання файлів:

```
git add .gitignore README.md scripts packs
```

Перший commit:

```
git commit -m "Add Xiaomi L07A UART sound pack guide"
```

Перейменувати branch у main:

```
git branch -M main
```

Додати remote:

```
git remote add origin https://github.com/vitaliyrenkas-rgb/Xiaomi-L07A-UART-SoundHack.git
```

Push:

```
git push -u origin main
```

Tag v0.1:

```
git tag l07a-soundpack-v0.1
```

```
git push origin l07a-soundpack-v0.1
```

Після додавання MIT License:

```
git add LICENSE README.md
```

```
git commit -m "Add MIT license"
```

```
git push
```

Tag v0.1.1:

```
git tag l07a-soundpack-v0.1.1
```

```
git push origin l07a-soundpack-v0.1.1
```

Підтверджений результат:

```
main branch pushed
tag l07a-soundpack-v0.1 pushed
tag l07a-soundpack-v0.1.1 pushed
```

25. License

Для репозиторію використано MIT License.

Важливе формулювання:

The repository contains only original documentation, helper scripts, and generated replacement system melodies.

It does not contain original Xiaomi firmware, original Xiaomi audio assets, NAND/rootfs dumps, device tokens, private keys, or serial-number-specific data.

Це важливо, щоб не розповсюджувати пропріетарні файли Xiaomi.

26. Підсумок

Підтверджений фінальний стан:

- UART debug connector працює.
- U-Boot доступний.
- Linux root shell отримується через тимчасовий `init=/bin/sh`.
- Rootfs є squashfs read-only.
- Secure boot / rootfs verification не ламались.
- `/dev/nand0p9` монтується як writable EXT4 UDISK.
- Активний sound pack задається через `/tmp/udisk/sound`.
- Заводський symlink був:

```
/tmp/udisk/sound -> /usr/share/sound-vendor/AiNiRobot
```

- Кастомний symlink став:

```
/tmp/udisk/sound -> /tmp/udisk/sound_custom
```

- Китайські голосові prompt-и успішно замінено на системні мелодії.
- NAND/rootfs не перепрошивався.
- Secure boot не обходився й не ламався.
- Рішення оформлено в GitHub repository з MIT License.

Фінальна суть методу:

не ламати підписаний rootfs, а використати штатний writable UDISK sound symlink.